

Introduction to Message Broker

What is a Message Broker?

A **message broker** is a software that helps different applications or services talk to each other by sending and receiving messages. Instead of services calling each other directly, they send messages to the broker, which then delivers them to the right place. This makes systems more flexible, scalable, and reliable. It also allows services to work independently and at their own pace. Message brokers are key in building event-driven or microservices-based systems.

Benefits

1. **Decoupling** Microservices
2. **Asynchronous** Communication among microservices
3. Fault-tolerance execution
4. **Batch processing**
5. **Transactions** Support
6. Cross-language communications
7. Handling **Long-Running Tasks** like Video processing (converting into different formats) Image resizing and optimization, Report generation.

REST vs. Message Broker

Aspect	REST API	Message Broker
Communication	Synchronous – client waits for a response	Asynchronous – sender and receiver operate independently
Coupling	Tightly coupled – direct service-to-service calls	Loosely coupled – interaction via broker
Scalability	Moderate – requires effort to scale horizontally	High – built for distributed, high-throughput environments
Fault Tolerance & Delivery Guarantee	Low – if receiver is down, request fails; no guaranteed delivery	High – messages can persist; supports retries and guaranteed delivery
Protocol	HTTP/HTTPS	AMQP, MQTT, STOMP, Kafka protocol, etc.

Message Format	JSON, XML (typically)	JSON, XML, Avro, Protobuf, etc.
Ordering Guarantees	No built-in support	Supported by many brokers
Backpressure Handling	Not supported	Supported via queues and flow control
Monitoring	Basic – via HTTP status codes and logs	Rich metrics – queue length, consumer lag, throughput, etc.
Statefulness	Stateless – each request is independent	Stateful – messages and queues persist across time
Event-Driven Support	Not native – needs polling or custom logic	Built-in – designed for event-driven architectures

Popular Message Brokers

Message Queue-Based Brokers:

- **RabbitMQ:** Lightweight, widely used broker supporting AMQP. Excellent for standard messaging and job queues.
- **ActiveMQ:** A powerful Java-based broker, supports JMS, AMQP, MQTT.
- **AmazonMQ:** Managed message broker service from AWS, supports ActiveMQ and RabbitMQ engines.
- **AWS SQS (Simple Queue Service):** Fully managed serverless message queuing service.

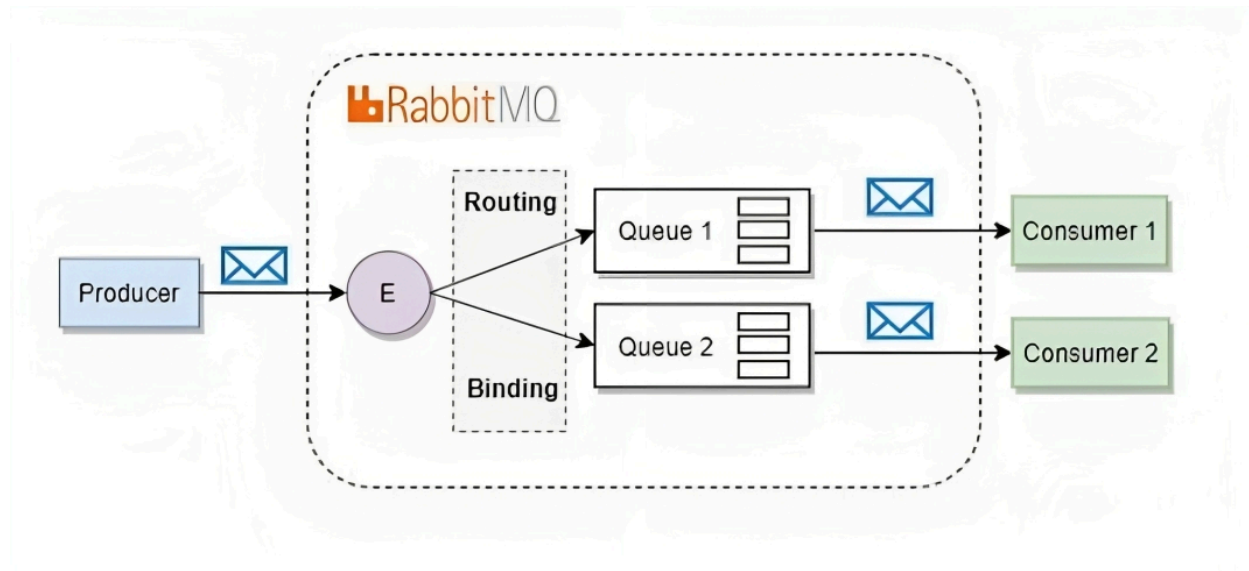
Stream Processing Brokers:

- **Apache Kafka:** Distributed event streaming platform for high-throughput, real-time data processing.
- **Redis Streams:** Lightweight stream processing within the Redis ecosystem.
- **RabbitMQ Streams:** Streaming support added to RabbitMQ for higher-throughput use cases.
- **Amazon Kinesis:** Fully managed service for real-time data streaming at scale.

RabbitMQ

RabbitMQ is an open-source **message broker** (also known as **message-oriented middleware**) that facilitates communication between distributed systems by allowing them to exchange messages asynchronously. It was originally built to implement the **Advanced Message Queuing Protocol (AMQP)** but has since evolved through a **pluggable architecture** to also support other messaging protocols such as **STOMP**, **MQTT**, and more.

RabbitMQ acts as a central hub that receives, stores, and routes messages between producers and consumers, enabling **decoupling**, **scalability**, and **reliability** in microservice architectures and event-driven systems.



Core components:

A. Producer

A producer is any application that sends (publishes) messages to a RabbitMQ exchange. Producers do not need to know who the consumers are or how messages are routed—this is handled by RabbitMQ.

B. RabbitMQ Server

The RabbitMQ server handles routing, the lifecycle of messages, queues, and client connections. It is the core broker service that mediates between producers and consumers.

B.1 Exchange

The exchange receives messages from producers and routes them to queues based on defined rules and bindings. There are 5 types:

- **Direct:** Sends messages to queues with an exact routing key match.
- **Fanout:** Broadcasts messages to all bound queues, regardless of routing key.
- **Topic:** Routes messages to queues based on pattern-matching (e.g., `logs.*.error`).
- **Headers:** Routes based on header values instead of routing keys.
- **Default:** A special type of direct exchange with an empty string ("") as its name, which routes messages to queues where the **routing key exactly matches the queue name**.

B.2 Routing

Routing in RabbitMQ determines how messages flow from an exchange to one or more queues. It is governed by:

- **Routing Key:** A string attached to a message by the producer. Exchanges (except fanout) use it to determine which queues to route the message to.
- **Bindings:** Associations between an exchange and a queue, often with a routing key or pattern.
- **Binding Key:** Used with direct and topic exchanges to match against the message's routing key.
- **Pattern Matching:** Used in topic exchanges, where special wildcard characters (`*`, `#`) allow flexible routing logic.
- **Header Matching:** Used in header exchanges where routing decisions are made based on message headers instead of keys.

A message is routed **only** to the queues that match the routing rules defined by the exchange type and bindings.

B.3 Queues

Queues are where messages are stored until a consumer retrieves them. Each queue is FIFO (first in, first out).

- Queues can be **durable**, meaning they survive broker restarts if declared as such.
- Messages in queues can have a **TTL (time-to-live)**, after which they expire and are dropped or forwarded.
- **Dead-letter exchanges (DLX)** can be configured to handle rejected or expired messages for further processing or inspection.

- Queues can also be **exclusive** (used by one connection only) or **auto-delete** (deleted when the last consumer unsubscribes).

C. Consumer

A consumer subscribes to queues and processes messages. It can acknowledge successful processing, reject and drop messages, or reject and requeue them for retry. Consumers can be single-threaded, multi-threaded, or horizontally scaled depending on workload.

RabbitMQ Installation

There are several ways to install and run RabbitMQ, such as:

- Installing the **RabbitMQ server** directly on your system
- Using **AmazonMQ** (AWS-managed RabbitMQ service)
- Running RabbitMQ in a **Docker container**

For Python development, several client libraries are available, including:

- **pika** – the most widely used and lightweight library
- **aio-pika** – an async-friendly wrapper around **pika** for asyncio applications

In this assignment, we used the **Docker rabbitmq:management image** to run RabbitMQ, and the **pika library** for message publishing and consumption.

```
rejo@rejo:~$ docker run -d --rm --name rabbitmq -p 5672:5672 -p 15672:15672 rabbitmq:4-management
bc94504b7516bff7738115e55d5352e04a037f5c2b96a44ba56f1e886af861c7
rejo@rejo:~$ docker container ls
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
bc94504b7516   rabbitmq:4-management              "docker-entrypoint.s..." 4 minutes ago  Up 4 minutes  4369/tcp, 5671/tcp, 0.0.0.0:5672->5672/tcp, 15671/tcp, 15691-15692/tcp, 25672/tcp, 0.0.0.0:15672->15672/tcp   rabbitmq
```

Task 1: Hello World

Producer code:

```
import pika

parameters = pika.ConnectionParameters("localhost")

with pika.BlockingConnection(parameters) as connection:
    channel = connection.channel()
    channel.queue_declare("hello")
    channel.basic_publish(exchange="", routing_key="hello", body="Hello world!")
    print(" [x] Sent 'Hello World!'")
```

Calling producer 3 times:

```
rejo@rejo:~/Desktop/message-broker/RabbitMQ-Hello-World
rejo@rejo:~/Desktop/message-broker/RabbitMQ-Hello-World$ python producer.py
[x] Sent 'Hello World!'
rejo@rejo:~/Desktop/message-broker/RabbitMQ-Hello-World$ python producer.py
[x] Sent 'Hello World!'
rejo@rejo:~/Desktop/message-broker/RabbitMQ-Hello-World$ python producer.py
[x] Sent 'Hello World!'
```

RabbitMQ received the messages successfully. The RabbitMQ Management UI is running at <http://localhost:15672/>

 RabbitMQ™

RabbitMQ 4.1.0 Erlang 27.3.3

Refreshed 2025-05-05 00:30:31 Refresh every 30 seconds

Virtual host All

Cluster rabbit@6459f1e8d370

User guest Log out

⚠️ Deprecated features are being used. [\[Learn more\]](#)

OverviewConnectionsChannelsExchangesQueues and StreamsAdmin

Queues

▼ All queues (1)

Page 1 of 1 - Filter: ☐ Regex ?

Displaying 1 item , page size up to: 100

Overview					Messages			Message rates			
Virtual host	Name	Type	Features	State	Ready	Unacked	Total	incoming	deliver / get	ack	
/	hello	classic	Args	running	3	0	3	0.00/s	0.00/s	0.00/s	+/-

Consumer code:

```
import pika
import os, sys, time

def callback(ch, method, properties, body):
    time.sleep(3) # Simulate a long-running task
    print(f" [x] Received: {body}")
    print(f"      Properties: {properties}")

def main():
    parameters = pika.ConnectionParameters("localhost")

    with pika.BlockingConnection(parameters) as connection:
        channel = connection.channel()

        channel.queue_declare("hello") # Idempotent func, will not overwrite existing Q
        channel.basic_consume(
            queue="hello",
            on_message_callback=callback,
            auto_ack=True,
        )

        print(" [*] Waiting for messages. To exit press CTRL+C")
        # Blocking loop; listens for messages
        # Runs on Main thread
        channel.start_consuming()

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("Interrupted")
        try:
            sys.exit(0)
        except SystemExit:
            os._exit(0)
```

Running consumer:

```
rejo@rejo: ~/Desktop/message-broker/RabbitMQ-Hello-World$ python consumer.py
[*] Waiting for messages. To exit press CTRL+C
[x] Received: b'Hello world!'
Properties: <BasicProperties>
[x] Received: b'Hello world!'
Properties: <BasicProperties>
[x] Received: b'Hello world!'
Properties: <BasicProperties>
```

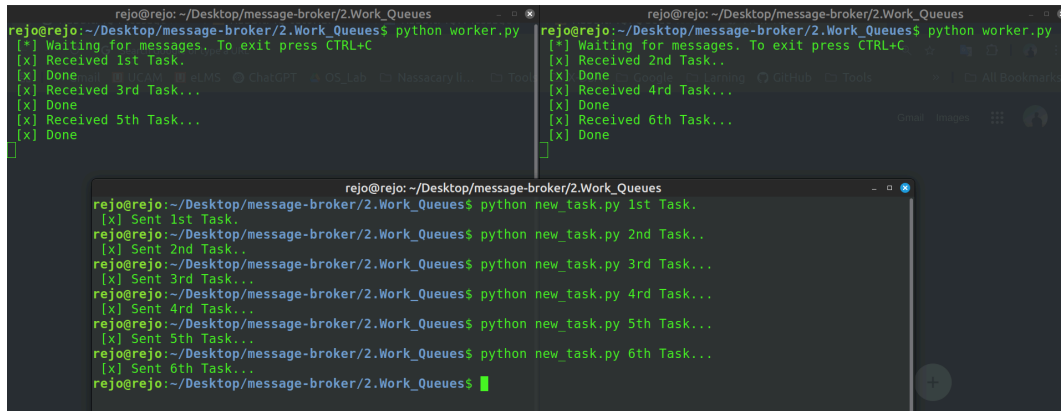
The consumer successfully received 3 messages and is now waiting for more.

The **channel.start_consuming()** function blocks the main thread and continuously listens for incoming messages. To enable concurrent processing, you can use Python's **threading** or **multiprocessing** module inside the callback function. This allows the consumer to handle multiple messages in parallel without waiting for each one to finish.

Task 2: Work Queues

1. Round robin dispatch:

The **new_task.py** file is essentially the same as our previous **producer.py**, except that it now allows command-line arguments to be passed. The **worker.py** file is based on the previous **consumer.py** code. In this case, we simulate long-running tasks using **time.sleep()**.



```
rejo@rejo: ~/Desktop/message-broker/2.Work_Queue$ python worker.py
[*] Waiting for messages. To exit press CTRL+C
[x] Received 1st Task.
[x] Done
[x] Received 3rd Task...
[x] Done
[x] Received 5th Task...
[x] Done

rejo@rejo: ~/Desktop/message-broker/2.Work_Queue$ python worker.py
[*] Waiting for messages. To exit press CTRL+C
[x] Received 2nd Task..
[x] Done
[x] Received 4rd Task...
[x] Done
[x] Received 6th Task...
[x] Done

rejo@rejo: ~/Desktop/message-broker/2.Work_Queue$ python new_task.py 1st Task.
[x] Sent 1st Task.
rejo@rejo: ~/Desktop/message-broker/2.Work_Queue$ python new_task.py 2nd Task..
[x] Sent 2nd Task..
rejo@rejo: ~/Desktop/message-broker/2.Work_Queue$ python new_task.py 3rd Task...
[x] Sent 3rd Task...
rejo@rejo: ~/Desktop/message-broker/2.Work_Queue$ python new_task.py 4rd Task...
[x] Sent 4rd Task...
rejo@rejo: ~/Desktop/message-broker/2.Work_Queue$ python new_task.py 5th Task...
[x] Sent 5th Task...
rejo@rejo: ~/Desktop/message-broker/2.Work_Queue$ python new_task.py 6th Task...
[x] Sent 6th Task...
rejo@rejo: ~/Desktop/message-broker/2.Work_Queue$
```

As shown in the figure, two workers are running, and the workload is distributed in a round-robin fashion. Dispatching message round-robin is the default behaviour of rabbitmq.

2. Message acknowledgment

The purpose of acknowledgment (ack) is to let RabbitMQ know that a message has been successfully received and processed by a consumer. There are two types of message acknowledgment: automatic acknowledgment and manual acknowledgment.

Our previous example uses automatic acknowledgment by setting **auto_ack=True**:

```
channel.basic_consume(
    queue="work_queue",
    on_message_callback=callback,
    auto_ack=True,
)
```

But there is a flaw, it allows RabbitMQ to delete the message as soon as it is sent to the worker. In case the worker fails before completing the task, the message will be lost. That's where manual acknowledgment becomes useful. Let's **remove** the **auto_ack** flag.

Manual acknowledgement

```
def callback(ch, method, properties, body):
    print(f" [x] Received {body.decode()}")
    time.sleep(body.count(b"."))
    print(" [x] Done")
    ch.basic_ack(delivery_tag=method.delivery_tag)
```

We run **two workers** and create seven **tasks (1–7)**. While Worker-2 is processing Task 3, we stop the worker using Ctrl+C. As a result, the RabbitMQ server reassigns all tasks that were being processed by Worker-2 to Worker-1, as shown in the figure below.

```

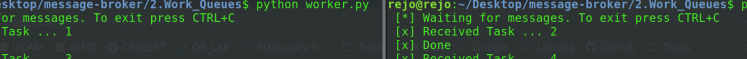
rejo:rejo:~/Desktop/message-broker/2.Work_Queue$ python worker.py
[*] Waiting for messages. To exit press CTRL+C
[x] Received Task .....2
[x] Done .....0
[x] Received Task .....4
[x] Done .....0
[x] Received Task .....7
[x] Done .....0
[x] Received Task .....3
[x] Done .....0
[x] Received Task .....5
[x] Done .....0
rejo:rejo:~/Desktop/message-broker/2.Work_Queue$ python worker.py
[*] Waiting for messages. To exit press CTRL+C
[x] Received Task .....1
[x] Done .....0
[x] Received Task .....3
[x] Done .....0
[9]+ Stopped python worker.py
rejo:rejo:~/Desktop/message-broker/2.Work_Queue$

```

Note: that Forgotten acknowledgment is a very serious bug.

```
# ch.basic ack(delivery tag=method.delivery tag) # Comment
```

RabbitMQ considers the message in-flight and **won't delete or redeliver it unless the worker disconnects. Memory use can grow.**



```
rejo@rejo: ~/Desktop/message-broker/2.Work_Queue$ python worker.py
[*] Waiting for messages. To exit press CTRL+C
[x] Received Task ... 1
[x] Done
[x] Received Task ... 2
[x] Received Task ... 3
[x] Done
[x] Received Task ... 4
[x] Done

rejo@rejo:~/Desktop/message-broker/2.Work_Queue$ python worker.py
[*] Waiting for messages. To exit press CTRL+C
[x] Received Task ... 2
[x] Done
[x] Received Task ... 3
[x] Received Task ... 4
[x] Done

rejo@rejo:~$ docker exec -it rabbitmq /bin/bash
root@6459f1e8d370:/# rabbitmqctl list_queues name messages_ready messages_unacknowledged
Timeout: 60.0 seconds ...
Listing queues for vhost / ...
name messages_ready messages_unacknowledged
work queues 0 4
root@6459f1e8d370:/#
```

3. Message durability

To ensure that messages are not lost if RabbitMQ is stopped or crashes, both the queue and the messages must be made durable.

Queue Durability (Required on both producer and consumer sides):

```
channel.queue declare("work queue", durable=True)
```

This ensures the queue itself survives a broker restart.

Message Durability (Producer side only):

```
channel.basic_publish(  
    exchange="",  
    routing_key="work_queue",  
    body=message,  
    properties= pika.BasicProperties(delivery_mode=2),  
)
```

There are two types of delivery modes:

1. (Transient): The default. Messages are not persisted.
2. (Persistent): Messages are saved to disk.

Important: Even with durable queues and persistent messages, messages can still be lost if RabbitMQ crashes before flushing them to disk. For full reliability, consider using publisher confirms using **`channel.confirm_delivery()`**.

4. Fair dispatch

By default, RabbitMQ distributes messages evenly without considering how busy each worker is. This can lead to some workers being overloaded while others sit idle. To fix this, you can **`channel.use basic_qos(prefetch_count=1)`**, which ensures a worker gets only one unacknowledged message at a time. RabbitMQ will then wait for a worker to finish processing before sending it another message. This helps distribute the load more fairly among workers.

5. Putting it all together

new_task.py

```
import sys
import pika

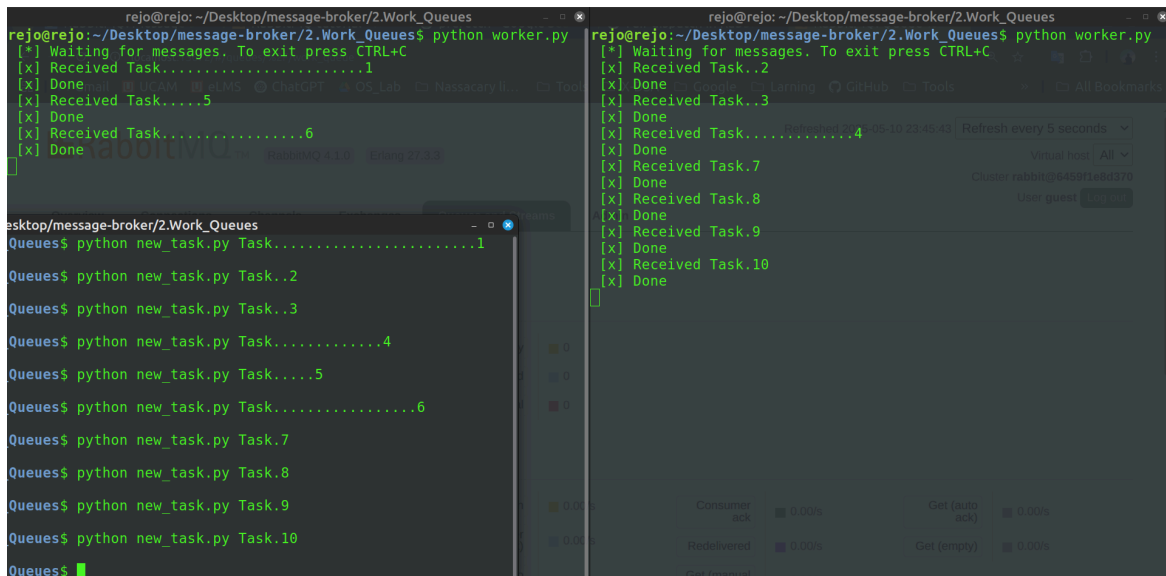
message = " ".join(sys.argv[1:]) or "Hello world"
conn_params = pika.ConnectionParameters("localhost")

with pika.BlockingConnection(conn_params) as connection:
    channel = connection.channel()
    channel.queue_declare("work_queue", durable=True)

    channel.basic_publish(
        exchange="",
        routing_key="work_queue",
        body=message,
        properties=pika.BasicProperties(delivery_mode=2),
    )

    print(f" [x] Sent {message}")
```

Output:



The screenshot displays two terminal windows. The left window shows the execution of `new_task.py` with arguments `Task..1` through `Task..10`. The right window shows the output of `worker.py`, which receives and processes these tasks. The output of `worker.py` includes a list of received tasks and their completion status, along with a table of performance metrics.

```
rejo@rejo: ~/Desktop/message-broker/2.Work_Queue$ python worker.py
[*] Waiting for messages. To exit press CTRL+C
[x] Received Task.....1
[x] Done
[x] Received Task.....2
[x] Done
[x] Received Task.....3
[x] Done
[x] Received Task.....4
[x] Done
[x] Received Task.....5
[x] Done
[x] Received Task.....6
[x] Done
[x] Received Task.....7
[x] Done
[x] Received Task.....8
[x] Done
[x] Received Task.....9
[x] Done
[x] Received Task.....10
[x] Done
```

Consumer	ack	Redelivered	Get (auto ack)	Get (empty)
0.0%	0.0%	0.0%	0.0%	0.0%

worker.py

```
import pika
import os, sys, time

def callback(ch, method, properties, body):
    print(f" [x] Received {body.decode()}")
    time.sleep(body.count(b"."))
    print(" [x] Done")
    ch.basic_ack(delivery_tag=method.delivery_tag)

def main():
    conn_params = pika.ConnectionParameters("localhost")

    with pika.BlockingConnection(conn_params) as connection:
        channel = connection.channel()
        channel.queue_declare("work_queue", durable=True)
        channel.basic_qos(prefetch_count=1) # fetch one message at a time
        channel.basic_consume(queue="work_queue", on_message_callback=callback)

        print(" [*] Waiting for messages. To exit press CTRL+C")
        channel.start_consuming()

if __name__ == "__main__":
    try:
        main()
    except KeyboardInterrupt:
        print("Interrupted")
        try:
            sys.exit(0)
        except SystemExit:
            os._exit(0)
```

Links:

1. <https://tsh.io/blog/message-broker/>
2. <https://www.youtube.com/watch?v=7rkeORD4jSw>
3. <https://www.rabbitmq.com/tutorials>